095946- ADVANCED ALGORITHMS AND PARALLEL PROGRAMMING



Fabrizio Ferrandi

a.a. 2026-2027

THE DIVIDE-AND-CONQUER DESIGN PARADIGM



- *Divide* the problem (instance) into subproblems.
- *Conquer* the subproblems by solving them recursively.
- *Combine* subproblem solutions.

Guiding question: how much non-recursive work happens at each recursion level?

DIVIDE AND CONQUER

- BINARY SEARCH
- POWERING A NUMBER
- MATRIX MULTIPLICATION
- STRASSEN'S ALGORITHM
- VLSI TREE LAYOUT

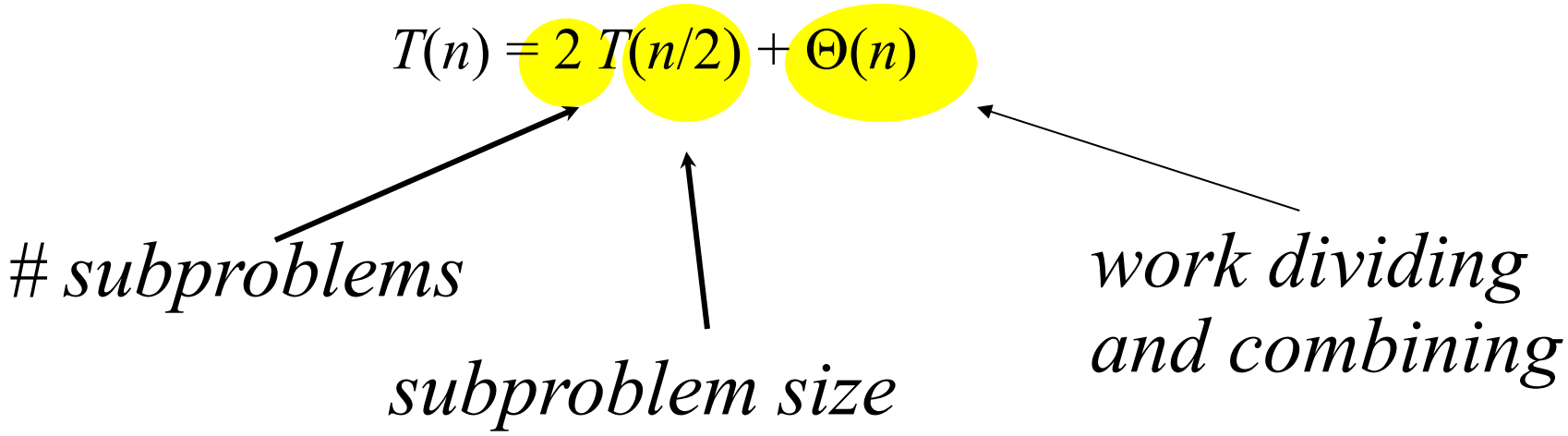


MERGE SORT

1. *Divide:* Trivial.
2. *Conquer:* Recursively sort 2 subarrays.
3. *Combine:* Linear-time merge.

MERGE SORT

1. *Divide:* Trivial.
2. *Conquer:* Recursively sort 2 subarrays.
3. *Combine:* Linear-time merge.



MASTER THEOREM (REPRISE)

$$T(n) = a\,T(n/b) + f(n)$$

CASE 1: $f(n) = O(n^{\log_b a - \varepsilon})$, constant $\varepsilon > 0$

$\Rightarrow T(n) = \Theta(n^{\log_b a})$.

CASE 2: $f(n) = \Theta(n^{\log_b a} \lg^k n)$, constant $k \geq 0$

$\Rightarrow T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.

CASE 3: $f(n) = \Omega(n^{\log_b a + \varepsilon})$, constant $\varepsilon > 0$, and regularity condition

$\Rightarrow T(n) = \Theta(f(n))$.

MASTER THEOREM (REPRISE)

$$T(n) = a\,T(n/b) + f(n)$$

CASE 1: $f(n) = O(n^{\log_b a - \varepsilon})$, constant $\varepsilon > 0$

$\Rightarrow T(n) = \Theta(n^{\log_b a})$.

CASE 2: $f(n) = \Theta(n^{\log_b a} \lg^k n)$, constant $k \geq 0$

$\Rightarrow T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.

CASE 3: $f(n) = \Omega(n^{\log_b a + \varepsilon})$, constant $\varepsilon > 0$, and regularity condition

$\Rightarrow T(n) = \Theta(f(n))$.

Merge sort: $a = 2$, $b = 2$ $\Rightarrow$ $n^{\log_b a} = n^{\log_2 2} = n$

$\Rightarrow$ CASE 2 ($k = 0$) $\Rightarrow$ $T(n) = \Theta(n \lg n)$.

BINARY SEARCH

Find an element in a sorted array:

1. *Divide:* Check middle element.
2. *Conquer:* Recursively search 1 subarray.
3. *Combine:* Trivial.

BINARY SEARCH

Find an element in a sorted array:

1. *Divide:* Check middle element.
2. *Conquer:* Recursively search 1 subarray.
3. *Combine:* Trivial.

Example: Find 9

3	5	7	8	9	12	15

BINARY SEARCH

Find an element in a sorted array:

1. *Divide:* Check middle element.
2. *Conquer:* Recursively search 1 subarray.
3. *Combine:* Trivial.

Example: Find 9



BINARY SEARCH

Find an element in a sorted array:

1. *Divide:* Check middle element.
2. *Conquer:* Recursively search 1 subarray.
3. *Combine:* Trivial.

Example: Find 9

3 5 7 8 9 12 15

BINARY SEARCH

Find an element in a sorted array:

1. *Divide:* Check middle element.
2. *Conquer:* Recursively search 1 subarray.
3. *Combine:* Trivial.

Example: Find 9



BINARY SEARCH

Find an element in a sorted array:

1. *Divide:* Check middle element.
2. *Conquer:* Recursively search 1 subarray.
3. *Combine:* Trivial.

Example: Find 9

3 5 7 8 9 12 15

BINARY SEARCH

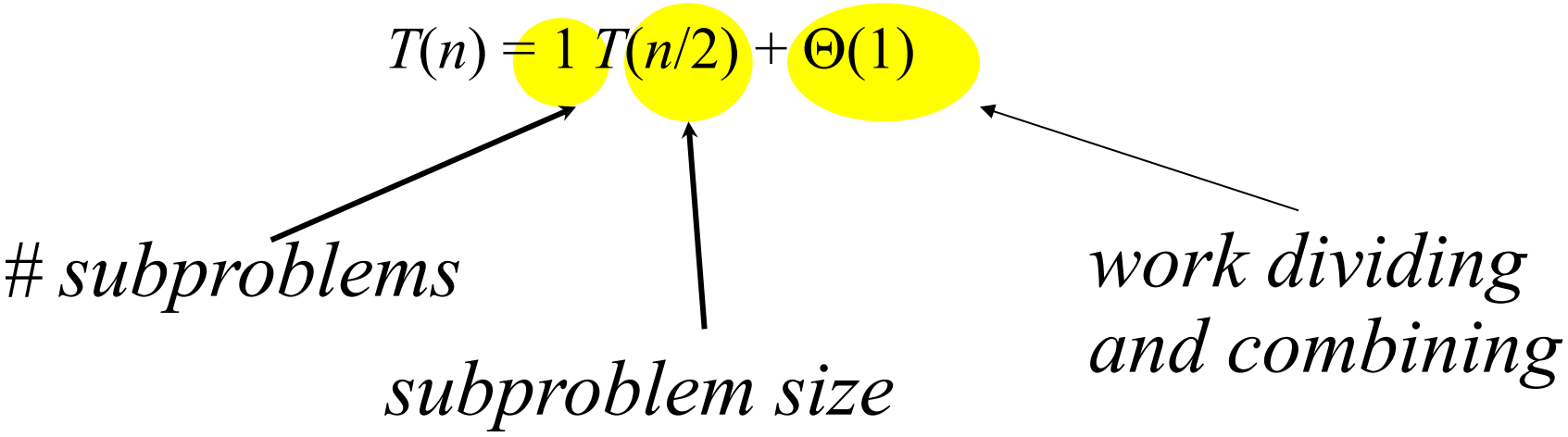
Find an element in a sorted array:

1. *Divide:* Check middle element.
2. *Conquer:* Recursively search 1 subarray.
3. *Combine:* Trivial.

Example: Find 9

3 5 7 8 9 12 15

RECURRENCE FOR BINARY SEARCH



RECURRENCE FOR BINARY SEARCH

$$T(n) = 1\,T(n/2) + \Theta(1)$$

subproblems

subproblem size

work dividing and combining

$n^{\log_b a} = n^{\log_2 1} = n^0 = 1 \Rightarrow$ CASE 2 ($k = 0$)

$\Rightarrow T(n) = \Theta(\lg n)$.

POWERING A NUMBER

Problem: Compute a^n, where $n \in N$.

Naive algorithm: $\Theta(n)$.

POWERING A NUMBER

Problem: Compute a^n, where $n \in N$.

Naive algorithm: $\Theta(n)$.

Divide-and-conquer algorithm:

$$a^n = \begin{cases} a^{n/2} \cdot a^{n/2} & \text{if } n \text{ is even;} \\ a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a & \text{if } n \text{ is odd.} \end{cases}$$

POWERING A NUMBER

Problem: Compute a^n, where $n \in N$.

Naive algorithm: $\Theta(n)$.

Divide-and-conquer algorithm:

$$a^n = \begin{cases} a^{n/2} \cdot a^{n/2} & \text{if } n \text{ is even;} \\ a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a & \text{if } n \text{ is odd.} \end{cases}$$

$$T(n) = T(n/2) + \Theta(1) \Rightarrow T(n) = \Theta(\lg n) .$$

MATRIX MULTIPLICATION

Input: $A = [a_{ij}], B = [b_{ij}].$

Output: $C = [c_{ij}] = A \cdot B.$

$\Big\}\ i, j = 1, 2, \ldots, n.$

$$\begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}$$

$$c_{ij} = \sum_{k=1}^{n} a_{ik} \cdot b_{kj}$$

STANDARD ALGORITHM

for $i \leftarrow 1$ to n
  do for $j \leftarrow 1$ to n
    do $c_{ij} \leftarrow 0$
      for $k \leftarrow 1$ to n
        do $c_{ij} \leftarrow c_{ij} + a_{ik} \cdot b_{kj}$

STANDARD ALGORITHM

for $i \leftarrow 1$ to n
  do for $j \leftarrow 1$ to n
    do $c_{ij} \leftarrow 0$
      for $k \leftarrow 1$ to n
        do $c_{ij} \leftarrow c_{ij} + a_{ik} \cdot b_{kj}$

Running time $= \Theta(n^3)$

DIVIDE-AND-CONQUER ALGORITHM

IDEA:

$n \times n$ matrix = 2×2 matrix of $(n/2) \times (n/2)$ submatrices:

$$\begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$C = A \cdot B$$

$$\left.\begin{aligned} r &= ae + bg \\ s &= af + bh \\ t &= ce + dg \\ u &= cf + dh \end{aligned}\right\} \begin{array}{l} 8 \text{ mults of } (n/2) \times (n/2) \text{ submatrices} \\ 4 \text{ adds of } (n/2) \times (n/2) \text{ submatrices} \end{array}$$

DIVIDE-AND-CONQUER ALGORITHM

IDEA:

$n \times n$ matrix = 2×2 matrix of $(n/2) \times (n/2)$ submatrices:

$$\begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$C = A \cdot B$$

$$\left.\begin{aligned} r &= ae + bg \\ s &= af + bh \\ t &= ce + dg \\ u &= cf + dh \end{aligned}\right\}$$

recursive

8 mults of $(n/2) \times (n/2)$ submatrices

4 adds of $(n/2) \times (n/2)$ submatrices

ANALYSIS OF D&C ALGORITHM

$$T(n) = 8\,T(n/2) + \Theta(n^2)$$

submatrices

submatrix size

work adding submatrices

ANALYSIS OF D&C ALGORITHM

$$T(n) = 8\,T(n/2) + \Theta(n^2)$$

submatrices

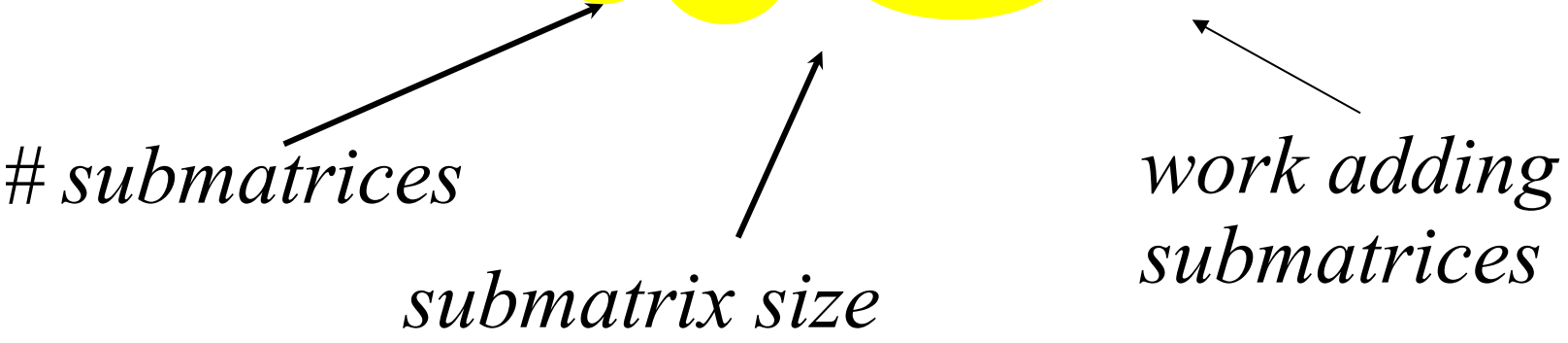
submatrix size

work adding submatrices

$$n^{\log_b a} = n^{\log_2 8} = n^3 \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^3).$$

ANALYSIS OF D&C ALGORITHM

$$T(n) = 8\,T(n/2) + \Theta(n^2)$$



$$n^{\log_b a} = n^{\log_2 8} = n^3 \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^3).$$

No better than the ordinary algorithm.

STRASSEN'S IDEA

- Multiply 2×2 matrices with only 7 recursive mults.

STRASSEN'S IDEA

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \cdot (f - h)$$
$$P_2 = (a + b) \cdot h$$
$$P_3 = (c + d) \cdot e$$
$$P_4 = d \cdot (g - e)$$
$$P_5 = (a + d) \cdot (e + h)$$
$$P_6 = (b - d) \cdot (g + h)$$
$$P_7 = (a - c) \cdot (e + f)$$

STRASSEN'S IDEA

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \cdot (f - h)$$
$$P_2 = (a + b) \cdot h$$
$$P_3 = (c + d) \cdot e$$
$$P_4 = d \cdot (g - e)$$
$$P_5 = (a + d) \cdot (e + h)$$
$$P_6 = (b - d) \cdot (g + h)$$
$$P_7 = (a - c) \cdot (e + f)$$

$$r = P_5 + P_4 - P_2 + P_6$$
$$s = P_1 + P_2$$
$$t = P_3 + P_4$$
$$u = P_5 + P_1 - P_3 - P_7$$

STRASSEN'S IDEA

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \cdot (f - h)$$
$$P_2 = (a + b) \cdot h$$
$$P_3 = (c + d) \cdot e$$
$$P_4 = d \cdot (g - e)$$
$$P_5 = (a + d) \cdot (e + h)$$
$$P_6 = (b - d) \cdot (g + h)$$
$$P_7 = (a - c) \cdot (e + f)$$

$$r = P_5 + P_4 - P_2 + P_6$$
$$s = P_1 + P_2$$
$$t = P_3 + P_4$$
$$u = P_5 + P_1 - P_3 - P_7$$

7 mults, 18 adds/subs.
Note: No reliance on commutativity of mult!

STRASSEN'S IDEA

- Multiply 2×2 matrices with only 7 recursive mults.

$$
\begin{aligned}
P_1 &= a \cdot (f - h) \\
P_2 &= (a + b) \cdot h \\
P_3 &= (c + d) \cdot e \\
P_4 &= d \cdot (g - e) \\
P_5 &= (a + d) \cdot (e + h) \\
P_6 &= (b - d) \cdot (g + h) \\
P_7 &= (a - c) \cdot (e + f)
\end{aligned}
$$

$$
\begin{aligned}
r &= P_5 + P_4 - P_2 + P_6 \\
&= (a + d)(e + h) \\
&\quad + d(g - e) - (a + b)h \\
&\quad + (b - d)(g + h) \\
&= ae + ah + de + dh \\
&\quad + dg - de - ah - bh \\
&\quad + bg + bh - dg - dh \\
&= ae + bg
\end{aligned}
$$

STRASSEN'S ALGORITHM

1. *Divide:* Partition A and B into $(n/2)\times(n/2)$ submatrices. Form terms to be multiplied using $+$ and $-$.
2. *Conquer:* Perform 7 multiplications of $(n/2)\times(n/2)$ submatrices recursively.
3. *Combine:* Form C using $+$ and $-$ on $(n/2)\times(n/2)$ submatrices.

STRASSEN'S ALGORITHM

1. *Divide:* Partition A and B into $(n/2)\times(n/2)$ submatrices. Form terms to be multiplied using $+$ and $-$.
2. *Conquer:* Perform 7 multiplications of $(n/2)\times(n/2)$ submatrices recursively.
3. *Combine:* Form C using $+$ and $-$ on $(n/2)\times(n/2)$ submatrices.

$$T(n) = 7\,T(n/2) + \Theta(n^2)$$

ANALYSIS OF STRASSEN

$$T(n) = 7\,T(n/2) + \Theta(n^2)$$

ANALYSIS OF STRASSEN

$$T(n) = 7\,T(n/2) + \Theta(n^2)$$

$n^{\log_b a} = n^{\log_2 7} \approx n^{2.81} \Rightarrow$ CASE 1 $\Rightarrow T(n) = \Theta(n^{\lg 7})$.

ANALYSIS OF STRASSEN

$$T(n) = 7\,T(n/2) + \Theta(n^2)$$

$n^{\log_b a} = n^{\log_2 7} \approx n^{2.81} \Rightarrow$ CASE 1 $\Rightarrow T(n) = \Theta(n^{\lg 7})$.

The number 2.81 may not seem much smaller than 3, but because the difference is in the exponent, the impact on running time is significant. In fact, Strassen's algorithm beats the ordinary algorithm on today's machines for $n \geq 32$ or so.

ANALYSIS OF STRASSEN

$$T(n) = 7\,T(n/2) + \Theta(n^2)$$

$n^{\log_b a} = n^{\log_2 7} \approx n^{2.81} \Rightarrow$ CASE 1 $\Rightarrow T(n) = \Theta(n^{\lg 7})$.

The number 2.81 may not seem much smaller than 3, but because the difference is in the exponent, the impact on running time is significant. In fact, Strassen's algorithm beats the ordinary algorithm on today's machines for $n \geq 32$ or so.

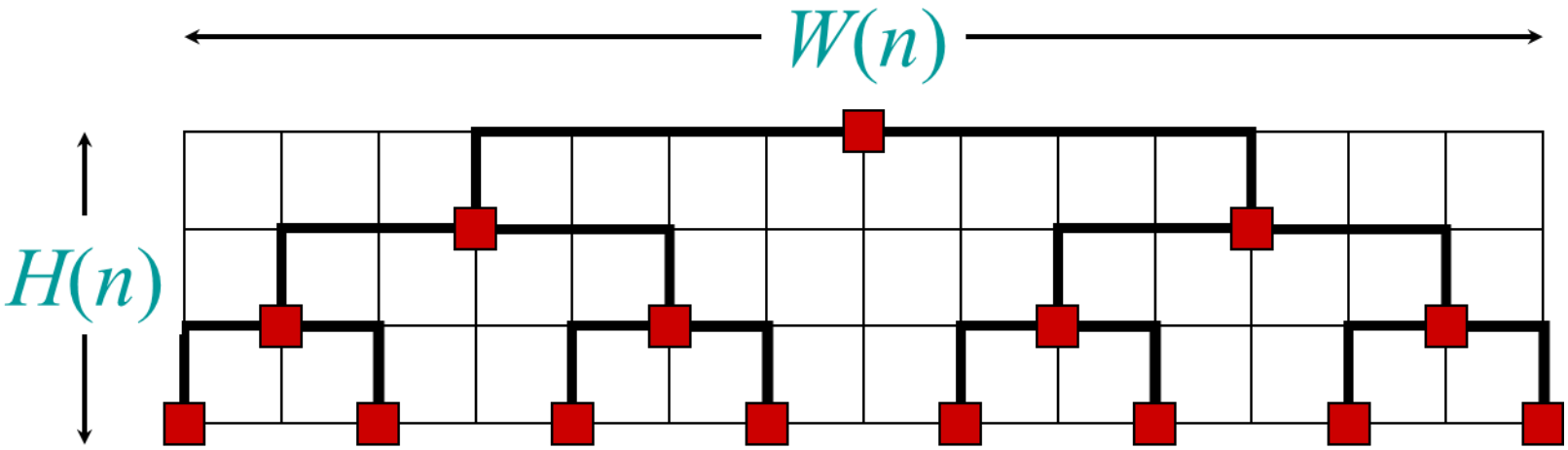
Best to date (of theoretical interest only): $\Theta(n^{2.376\cdots})$.

VLSI LAYOUT

Problem: Embed a complete binary tree with n leaves in a grid using minimal area.

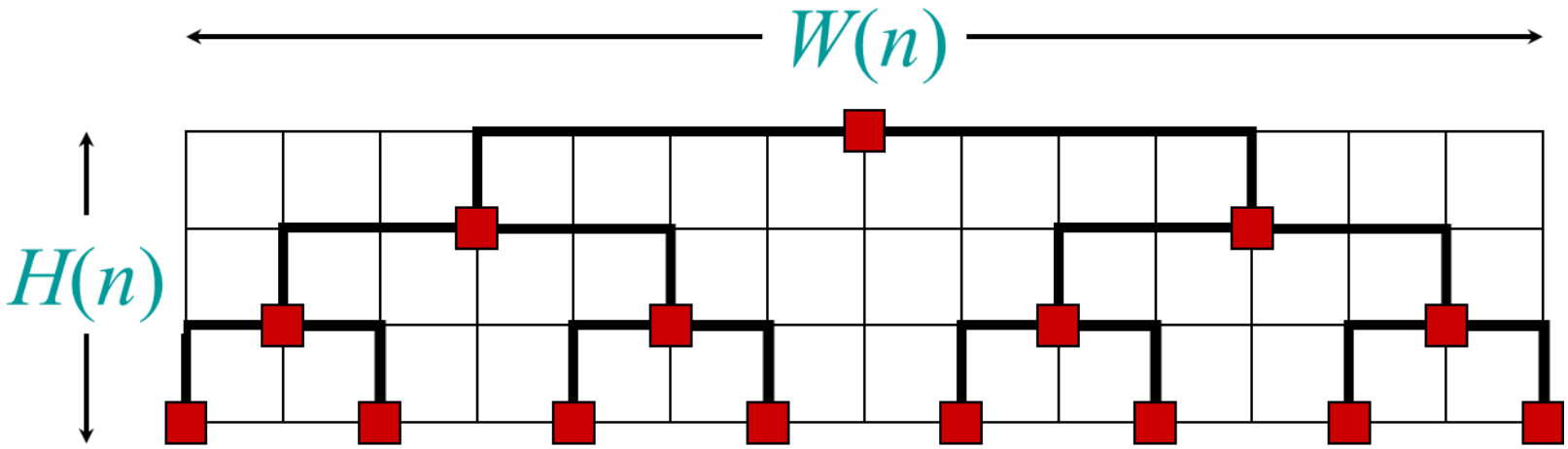
VLSI LAYOUT

Problem: Embed a complete binary tree with n leaves in a grid using minimal area.



VLSI LAYOUT

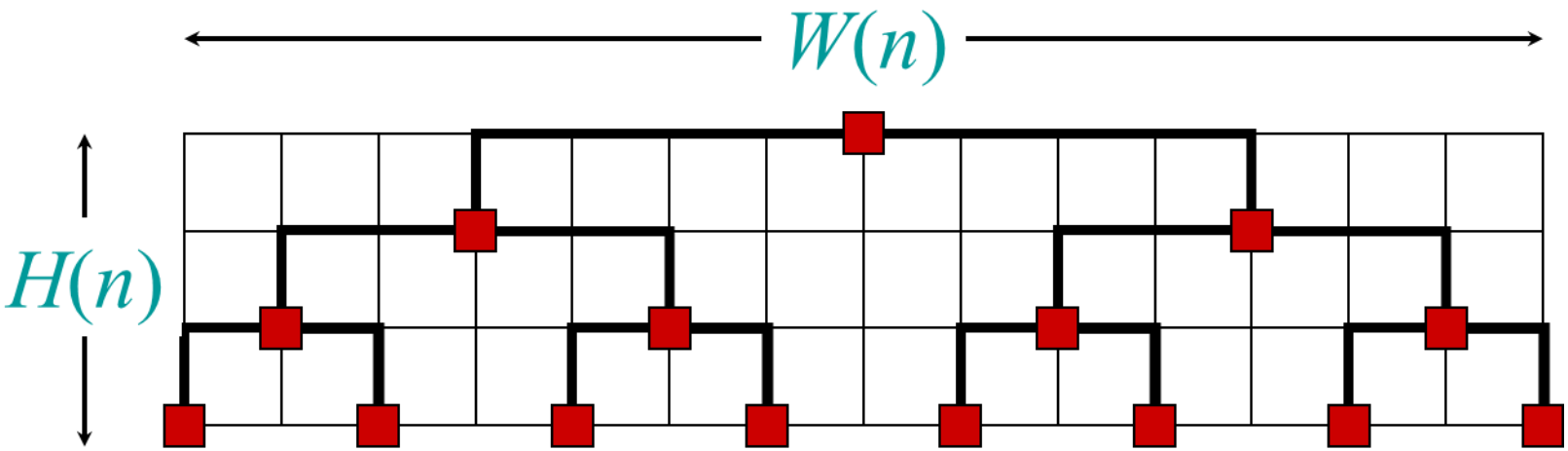
Problem: Embed a complete binary tree with n leaves in a grid using minimal area.



$$H(n) = H(n/2) + \Theta(1)$$
$$= \Theta(\lg n)$$

VLSI LAYOUT

Problem: Embed a complete binary tree with n leaves in a grid using minimal area.

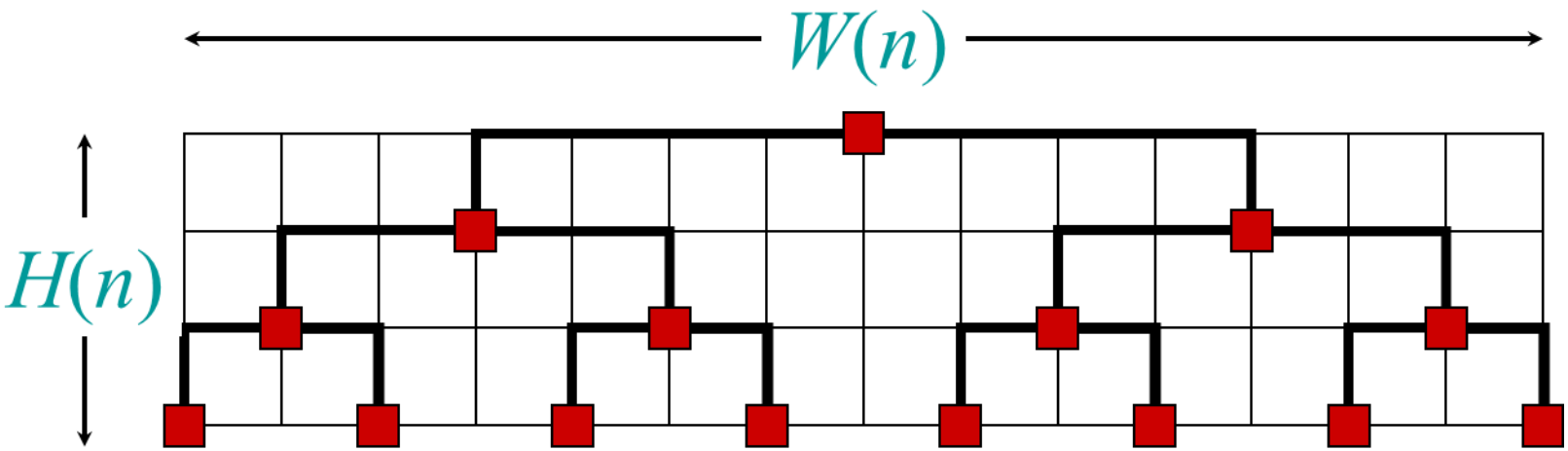


$$H(n) = H(n/2) + \Theta(1) = \Theta(\lg n)$$

$$W(n) = 2\,W(n/2) + \Theta(1) = \Theta(n)$$

VLSI LAYOUT

Problem: Embed a complete binary tree with n leaves in a grid using minimal area.

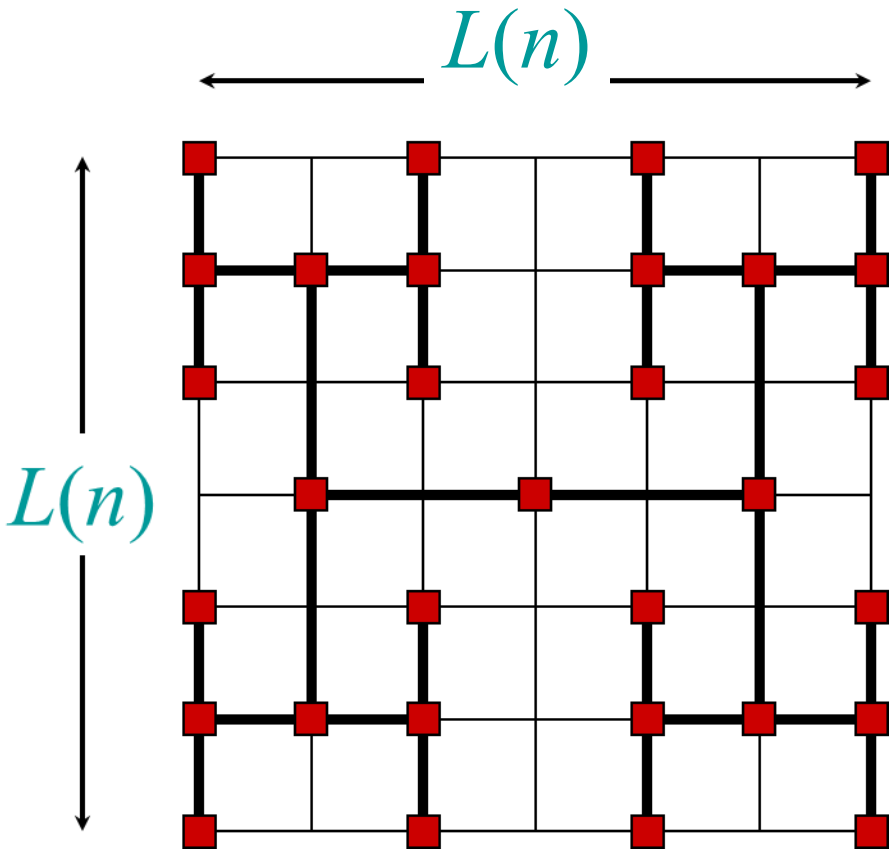


$$H(n) = H(n/2) + \Theta(1) = \Theta(\lg n)$$

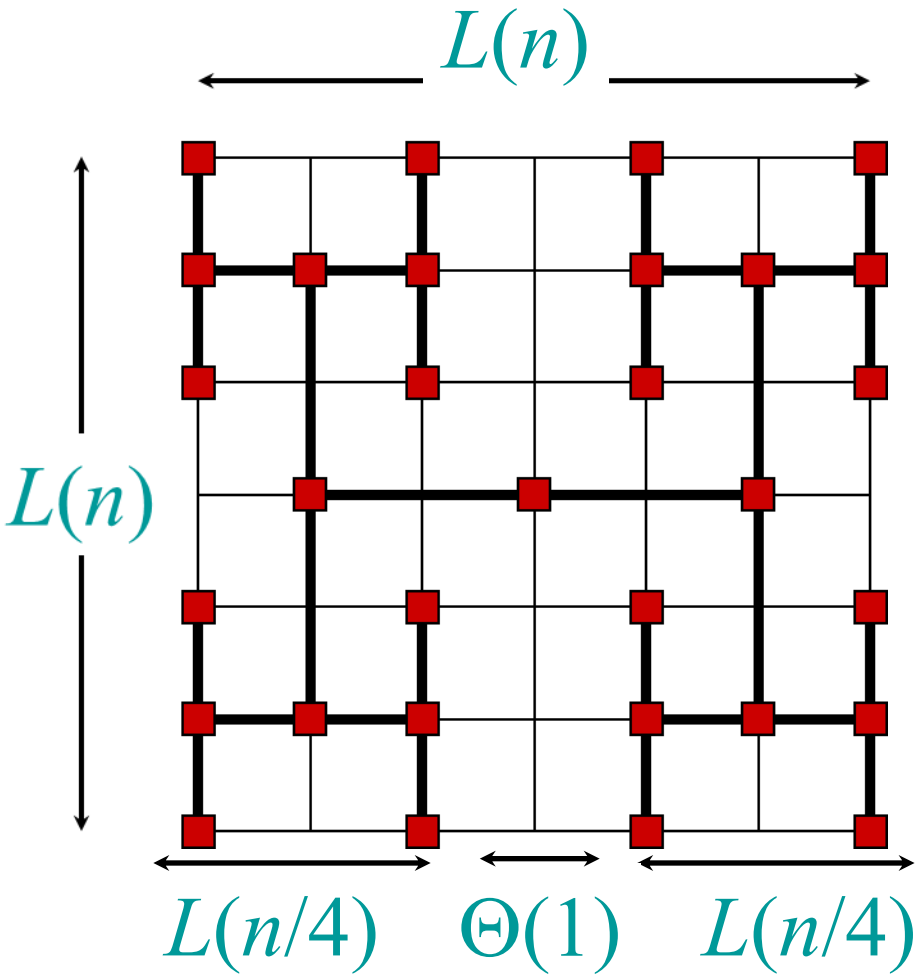
$$W(n) = 2\,W(n/2) + \Theta(1) = \Theta(n)$$

$$\text{Area} = \Theta(n \lg n)$$

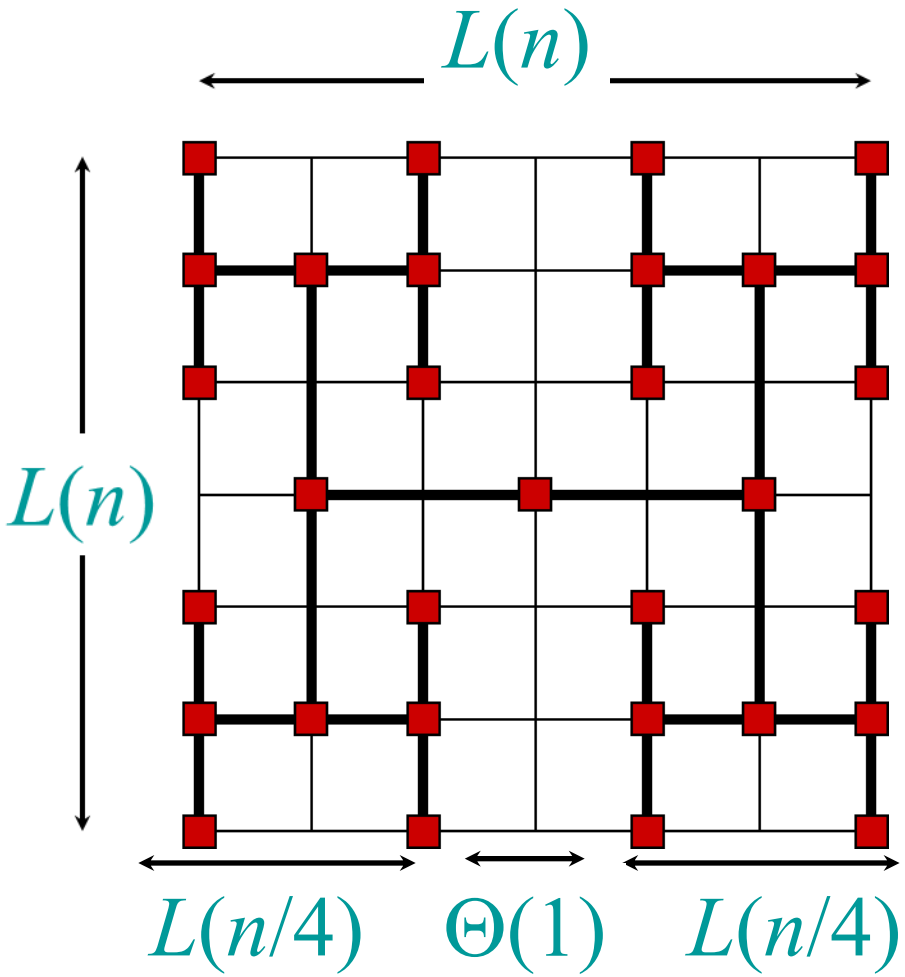
H-TREE EMBEDDING



H-TREE EMBEDDING



H-TREE EMBEDDING



$$L(n) = 2\,L(n/4) + \Theta(1)$$
$$= \Theta(\sqrt{n})$$

$$\text{Area} = \Theta(n)$$

CONCLUSION

- Divide and conquer is just one of several powerful techniques for algorithm design.
- Divide-and-conquer algorithms can be analyzed using recurrences and the master method (so practice this math).
- The divide-and-conquer strategy often leads to efficient algorithms.

Material adapted from Erik D. Demaine and Charles E. Leiserson slides